

| 48 Управление состоянием. Context API, основы Redux Toolkit

| Почему useEffect и useState не хватает

Аспект	Проблема при использовании useState + useEffect	Последствия для проекта
Пропс-드릴линг (Props Drilling)	Необходимость передавать состояние и колбэки через множество промежуточных компонентов, которые сами это состояние не используют	Код становится громоздким, сложным в поддержке и рефакторинге
Глобальное/общее состояние	Состояние, которое требуется в разных, часто удалённых частях приложения (корзина, данные пользователя, тема оформления, фильтры, состояние UI)	Раздувание контекста компонентов, нарушение принципа единственной ответственности
Производительность	При каждом изменении состояния перерисовываются все компоненты, подписанные на это состояние, даже если они используют только часть данных	Лишние рендеры, падение производительности при частых обновлениях (поиск, фильтрация, обновление корзины и т.д.)
Сложность логики обновлений	Множество useEffect с длинными зависимостями, race conditions, сложная синхронизация между несколькими кусками состояния	Трудноотлаживаемый код, баги, связанные с порядком обновлений и жизненным циклом
Масштабируемость	Локальное состояние разрастается и «вылезает» на верхние уровни приложения	Кодовая база теряет гибкость, усложняется тестирование и командная разработка

Хуки useState и useEffect часто не хватает для сложных задач, так как они создают «спагетти-код» из эффектов, приводят к лишним перерисовкам, не подходят для кэширования вычислений и не управляют состоянием на уровне всего приложения. Когда одно состояние зависит от изменения другого, код превращается в «лестницу» из useEffect. Это ухудшает читаемость и провоцирует бесконечные циклы, если зависимости указаны неверно. При каждом рендере React пересчитывает функции и массивы. Если у вас есть тяжелые вычисления, завязанные на useState, приложение будет зависать.

Для различного контекста проблем есть различные решения.

Redux Toolkit и **Context API** созданы для решения одной главной проблемы — избавления от «prop drilling» (когда данные нужно передавать через множество промежуточных компонентов). Однако они подходят для совершенно разных задач и масштабов приложения.

| Context API

React Context API — это встроенный инструмент React, который позволяет передавать данные (состояние, функции, настройки) напрямую в глубоко вложенные компоненты, минуя промежуточные звенья. Это избавляет от необходимости вручную передавать пропсы (props) через десятки компонентов. Вместо того чтобы пробрасывать пропсы через всю цепочку дерева компонентов, Context API позволяет объявить данные один раз на верхнем уровне и "доставать" их в любом месте приложения.

Может использоваться для:

- текущей темы приложения;
- данных авторизации пользователя;

- выбранного языка;
- фильтры;
- глобальных настроек приложения;
- корзины (практическое занятие №10).

Создание контекста, провайдер и другое

Контекст в React можно сравнить с глобальным хранилищем данных, к которому может получить доступ любой компонент, независимо от того, насколько глубоко в дереве он находится.

Создание контекста в React обеспечивается с помощью встроенной функции `createContext`. `createContext` создает объект контекста для обмена данными между компонентами. Она позволяет передавать информацию (глобальное состояние, темы оформления, язык) по всему дереву компонентов, избегая пропс-드릴линга.

```
1  const SomeContext = createContext(defaultValue)
```

Context.Provider — это встроенный компонент, который передает данные («контекст») всем дочерним компонентам (путем оборачивания дочерних компонентов в Provider).

Сейчас его можно использовать просто `<SomeContext>` как провайдер, раньше это было `<SomeContext.Provider>`.

```
1  return (
2
3    <CartContext.Provider
4      value={{ items, totalAmount, totalCount, addItem, updateQuantity, removeItem, clearCart }}
5    >
6      {children}
7    </CartContext.Provider>
8  );
```

```
1  function App() {
2    const [theme, setTheme] = useState('light');
3    // ...
4    return (
5      <ThemeContext value={theme}>
6        <Page />
7      </ThemeContext>
8    );
9  }
```

Чтением контекста раньше занимался `SomeContext.Consumer`. **Consumer** — это встроенный React-компонент, который используется для чтения данных из Context API. Он позволяет дочерним компонентам подписываться на изменения глобальных данных (например, темы оформления, данных пользователя или языка), не передавая их вручную через пропсы.

Компонент Consumer работает в паре с Provider (поставщиком). Provider задает значение, а Consumer считывает его, используя специальный паттерн `render prop`.

```
1  <SomeContext.Consumer>
2    {value => <div>Текущее значение: {value}</div>}
3  </SomeContext.Consumer>
4
```

Он использовался ранее, до того, как появился **useContext** — встроенный хук в библиотеке React, который позволяет компонентам читать данные и подписываться на глобальные переменные (контекст), не передавая их вручную через props на каждом уровне дерева компонентов.

```
1 import { useContext } from 'react';
2 import { ThemeContext } from './ThemeContext';
3
4 const MyComponent = () => {
5   const theme = useContext(ThemeContext);
6   return <div className={`box-${theme}`}>Текущая тема: {theme}</div>;
7 };
8
```

Использование нескольких контекстов в React позволяет разделить логику приложения на независимые модули (например, темы оформления, данные пользователя, корзину товаров), избежать лишних перерисовок и сделать код более поддерживаемым. Т.е. можно сделать и контекст для темы, и для пользователя, и для корзины одновременно.

```
1 import { createContext } from 'react';
2
3 //Контекст для темы оформления
4 export const ThemeContext = createContext();
5
6 //Контекст для данных пользователя
7 export const UserContext = createContext();
8
9 //Контекст для корзины
10 export const CartContext = createContext();
11
```

Также, внутри компонента, можно вызывать `useContext` столько раз, сколько необходимо.

```
1 import React, { useContext } from 'react';
2 import { ThemeContext, UserContext } from './contexts';
3
4 function UserProfile() {
5   const { theme, setTheme } = useContext(ThemeContext);
6   const { user } = useContext(UserContext);
7
8   return (
9     <div style={{ background: theme === 'dark' ? '#333' : '#fff' }}>
10       <p>Привет, {user.name}</p>
11       <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
12         Сменить тему
13       </button>
14     </div>
15   );
16 }
17
```

Главные правила использования:

- порядок провайдеров: порядок вложенности провайдеров не имеет значения для хука `useContext`;
- оптимизация: каждый Provider обновляет все компоненты, использующие его через `useContext`; разделение на несколько контекстов предотвращает лишние перерисовки, так как компонент будет обновляться только при изменении того контекста, на который он подписан;
- кастомные хуки: чтобы не импортировать `useContext` и сам контекст каждый раз, можно создать свой хук.

```

1 // В файле useContext.js
2 export const useUser = () => useContext(UserContext);
3
4 // В компоненте:
5 const { user } = useUser();
6

```

Проблемы и ограничения Context API

Основными проблемами являются:

- каскадные рендеры: любое изменение значения в Provider приводит к повторному рендерингу всех компонентов-потребителей (consumers), подписанных на этот контекст, даже если они используют только малую часть данных;
- неэффективность `React.memo`: оборачивание компонентов в `React.memo` не спасает от ререндера, если изменяется проп, передаваемый через контекст;
- отсутствие селекторов: Context API заставляет компоненты забирать весь объект целиком, нельзя подписаться только на одно поле объекта так, чтобы остальные изменения игнорировались;
- сложность масштабирования: по мере роста проекта логика провайдеров разрастается («Ад провайдеров» / Context Hell), код становится трудночитаемым из-за глубокой вложенности тегов;
- передача новых ссылок: если не использовать `useMemo` для значения `value` в Provider, то при каждом ререндере родительского компонента будет создаваться новый объект, что вызовет принудительный ререндер всех подписанных компонентов.

Решения проблем Context API

Zustand — это небольшая, быстрая и масштабируемая библиотека для управления состоянием (state management) в React-приложениях. Она использует хуки для управления данными, не требует оборачивания приложения в провайдеры (Context API) и содержит минимальное количество шаблонного кода.

Ссылка на GitHub-репозиторий: <https://github.com/pmndrs/zustand>



Jotai — это популярная и легковесная библиотека управления глобальным состоянием (state manager) для приложений React. В переводе с японского её название означает «состояние». Вместо того чтобы хранить все данные в одном громоздком хранилище, Jotai использует атомарный подход (снизу вверх).

Ссылка на GitHub-репозиторий: <https://github.com/pmndrs/jotai>



状態

Jōtai ^{v2}

Primitive and flexible state
management for React

Redux — это популярная JavaScript-библиотека, предназначенная для удобного и предсказуемого управления состоянием приложения. Она гарантирует, что все данные (например, профиль пользователя или товары в корзине) хранятся в одном месте и меняются по строгим правилам, что облегчает отладку и разработку сложных интерфейсов.

Ссылка на GitHub-репозиторий: <https://github.com/reduxjs/redux>



Redux

| Redux

Redux

A JS library for predictable and maintainable global state management

[Get Started](#)


Predictable

Redux helps you write applications that **behave consistently**, run in different environments (client, server, and native), and are **easy to test**.



Centralized

Centralizing your application's state and logic enables powerful capabilities like **undo/redo**, **state persistence**, and much more.



Debuggable

The Redux DevTools make it easy to trace **when, where, why, and how** your application's state changed. Redux's architecture lets you log changes, use **"time-travel debugging"**, and even send complete error reports to a server.



Flexible

Redux **works with any UI layer**, and has a **large ecosystem of addons** to fit your needs.

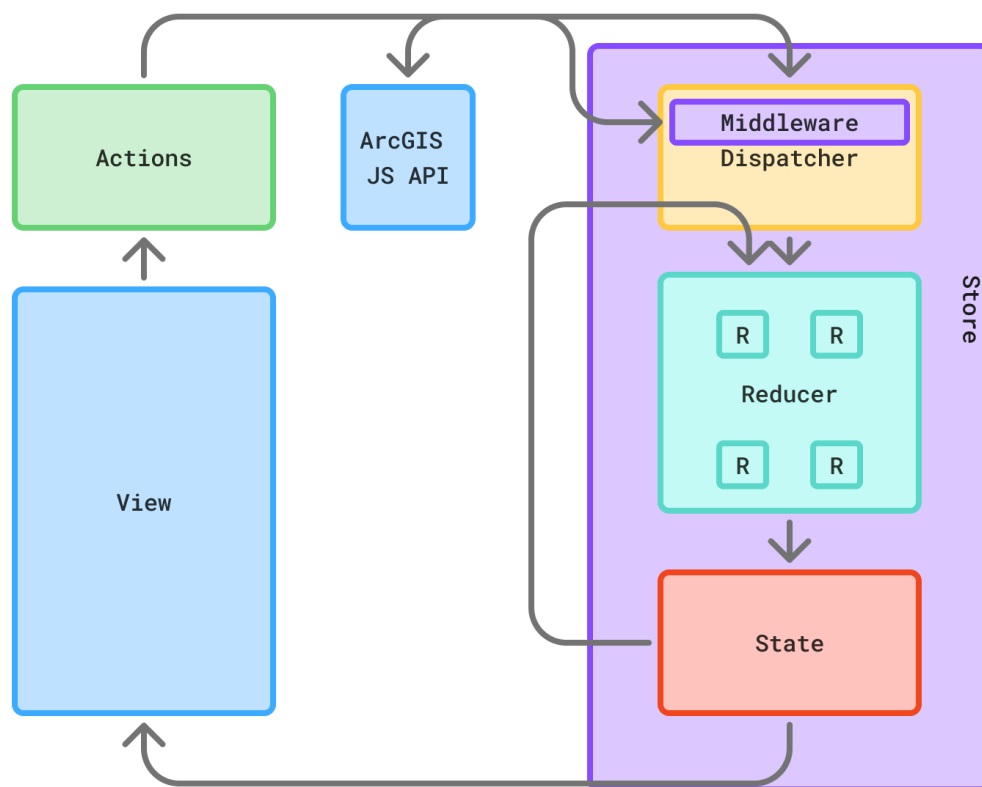


A NEW COURSE BY MARK ERIKSON + UI.DEV

[LEARN MORE](#)

Redux — как было сказано ранее, это JS-библиотека для предсказуемого и удобного управления глобальным состоянием.

Она помогает создавать приложения, которые ведут себя единообразно, работают в разных средах (на стороне клиента, сервера и в нативных приложениях) и легко тестируются. Кроме того, она предоставляет разработчикам множество удобных функций, таких как редактирование кода в реальном времени в сочетании с отладчиком, позволяющим перемещаться во времени. Redux можно использовать вместе с React или любой другой библиотекой для создания представлений.



На схеме изображена архитектура Redux (однаправленный поток данных) с интеграцией внешней библиотеки ArcGIS JS API для работы с географическими картами (как пример).

Данные в этой системе движутся строго по кругу в одном направлении:

1. Пользователь взаимодействует с интерфейсом;
2. Интерфейс генерирует событие;
3. Действие отправляется в диспетчер;
4. По пути действие может запустить сторонний сервис (API) или сразу пойти в обработчик;
5. Reducer обновляет состояние;
6. View узнает об изменении State и перерисовывается.

View (представление) — интерфейс приложения (React-компоненты), который видит пользователь.

Actions (действия) — простые JavaScript-объекты, которые описывают, что именно произошло (например, клик по карте или загрузка данных).

Store (хранилище) — единый центр управления данными, объединяющий диспетчер, редюсеры и состояние.

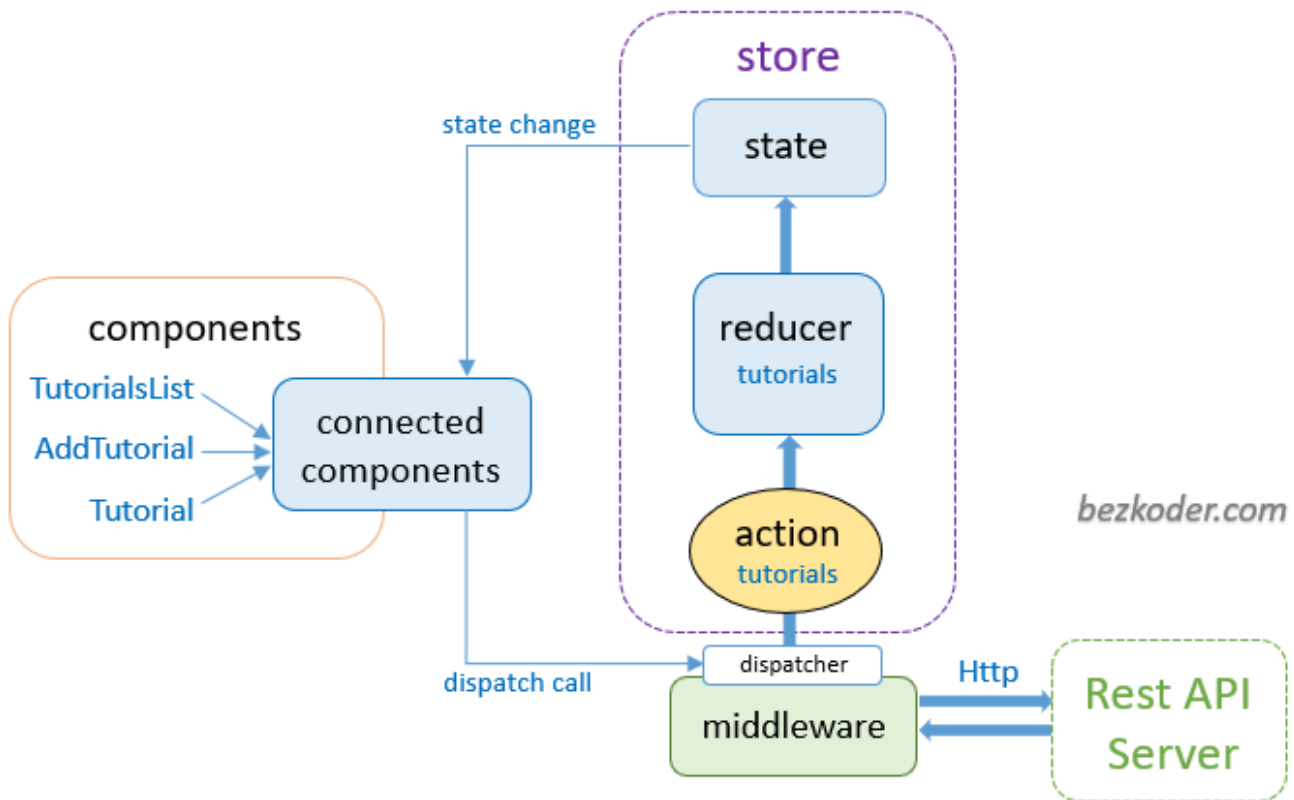
Reducer (редюсер) — чистая функция, которая принимает текущее состояние и действие, а затем возвращает новое состояние. Внутри содержит мелкие редюсеры (R).

State (состояние) — единственный источник правды. Объект, где хранятся все текущие данные приложения.

Больше о данных составляющих: <https://redux.js.org/tutorials/fundamentals/part-3-state-actions-reducers>

Redux Toolkit

Redux Toolkit (RTK) — это официальный, рекомендуемый набор инструментов для эффективной разработки на Redux. Он упрощает настройку хранилища (store), создание редьюсеров и иммутабельное обновление состояния, решая проблему избыточного кода и сложной настройки классического Redux.



Flux — это архитектурный паттерн (набор правил), разработанный Facebook для управления состоянием веб-приложений. Его главная особенность — строго однонаправленный поток данных (то, что было объяснено выше).

В классических паттернах (вроде MVC) связи часто двунаправленные, из-за чего изменения в одной части системы вызывают каскадные и труднопредсказуемые обновления в другой. Во Flux данные всегда движутся только по кругу: Action — Dispatcher — Store — View — Action. Изменения состояния не происходят спонтанно. Чтобы обновить UI, разработчик обязан создать Action. Это делает поведение приложения предсказуемым и значительно упрощает отладку.

Redux взял за основу идеи **Flux**, но упростил их, сделав архитектуру более строгой. Во Flux было несколько хранилищ и диспетчер, а Redux использует единый глобальный объект состояния и обходится без диспетчера.

Основные возможности Redux Toolkit

Возможность / принцип	Что даёт	Практический смысл	Пример
Единое хранилище данных	Все состояние приложения сосредоточено в одном месте	Проще контролировать изменения и отлаживать поведение приложения	Вся информация о пользователе, корзине и настройках хранится в одном хранилище.

Возможность / принцип	Что даёт	Практический смысл	Пример
Предсказуемое изменение состояния	Состояние изменяется только через действия и редьюсеры	Логика проще понимать, тестировать и сопровождать	Нажали кнопку — отправили действие, редьюсер обновил счётчик.
<code>configureStore</code>	Быстрая и стандартизированная настройка хранилища	Автоматически подключает типовые настройки	<pre>const store = configureStore({ reducer: { cart: cartReducer } })</pre>
<code>createSlice</code>	Объединяет состояние, редьюсеры и генерацию действий	Уменьшает объём кода	В одном файле описываются <code>initialState</code> , <code>addItem</code> , <code>removeItem</code> .
Иммутабельное обновление с удобным синтаксисом	Можно писать код в «изменяющем» стиле, а внутри сохраняется неизменяемость	Удобнее обновлять вложенные структуры	<code>state.items.push(newItem)</code> вместо ручного копирования массива.
Асинхронная логика	Упрощает работу с запросами и другими асинхронными операциями	Удобно описывать загрузку, успех и ошибку	Загрузка списка товаров с сервера и сохранение статусов <code>loading / succeeded / failed</code> .
Дополнительные обработчики	Один модуль состояния может реагировать на действия из других модулей	Полезно при сбросе данных или общих событиях	После выхода пользователя очищается корзина и профиль.
Селекторы	Позволяют удобно получать производные данные из хранилища	Снижают дублирование вычислений	Получить только товары из корзины, а не всё состояние целиком.
Поддержка разработческих проверок	Помогает находить ошибки вроде случайного изменения состояния	Повышает надёжность приложения	При попытке изменить объект состояния напрямую появляется предупреждение.
Интеграция с инструментами разработки	Видно, какие действия были отправлены и как менялось состояние	Ускоряет поиск ошибок	Можно отследить, что именно привело к изменению счётчика.
Работа с нормализованными данными	Удобнее хранить сущности в структурированном виде	Особенно полезно для списков и связей между объектами	Пользователи хранятся по идентификатору, а не как вложенный массив.
RTK Query	Встроенное решение для получения и кэширования данных	Подходит для серверных запросов без лишнего кода	<code>useGetPostsQuery()</code> получает данные и автоматически кэширует результат.

Immer.js — это популярная JavaScript-библиотека, которая позволяет удобно и безопасно работать с неизменяемыми (иммутабельными) данными. Она позволяет писать код, который выглядит как прямое изменение (мутация) данных, но на деле создает новый, обновленный объект, защищая оригинал. В таких библиотеках и фреймворках, как React или Redux, состояние должно быть иммутабельным. Это значит, что нельзя просто изменить свойство объекта, так как приложение этого не заметит. С Immer можно менять данные напрямую, а библиотека сама позаботится о создании новой копии объекта за кулисами.

createAsyncThunk — это функция из библиотеки Redux Toolkit, которая предназначена для обработки асинхронных операций (например, запросов к API) и генерации Redux-действий. Она упрощает написание асинхронного кода, избавляя от необходимости вручную создавать действия для каждого этапа запроса.

📌 Ссылки на документацию и tutorиалы

1. Redux Toolkit — <https://redux-toolkit.js.org/>;
2. Redux — <https://redux.js.org/>;
3. Redux Toolkit GitHub-репозиторий — <https://github.com/reduxjs/redux-toolkit>;
4. React Redux Toolkit Tutorial For Beginners - Redux Toolkit CRUD — https://www.youtube.com/watch?v=QgK_-G-hWeA;
5. React Redux Toolkit Projects For Beginners — <https://www.youtube.com/watch?v=R5xYw5kmh9k&list=PLN7bty2MYPQHh6PKiQ7GuVvJ1Smwlp1LN>.